

Technical Documentation — AI-Powered Smart Recruitment Assistant

This document walks through the notebook `AI_Powered_Smart_Recruitment_Assistant_.ipynb` section by section, explaining what each part of the code does and why.

1. Imports

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report, confusion_matrix,
                             roc_auc_score, roc_curve)

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
```

Standard data-science stack: `pandas` / `numpy` for data handling, `scikit-learn` for preprocessing/modeling/metrics, `matplotlib` / `seaborn` for visualization.

2. Loading the dataset

```
df_train = pd.read_csv('aug_train.csv')
df_test = pd.read_csv('aug_test.csv')
df_train['source'] = 'train'
df_test['source'] = 'test'
df = pd.concat([df_train, df_test], ignore_index=True)
```

The train and test files are tagged with a `source` column and concatenated into a single frame `df`. This lets cleaning/encoding be written **once** and applied consistently to both, and the `source` flag is used later to split them back apart.

Resulting shape after concatenation: **21,287 rows × 12 columns** (after the drop step below).

3. Dropping unused columns

```
df = df.drop(['enrollee_id', 'city', 'gender'], axis=1)
```

- `enrollee_id` — a unique identifier, no predictive value.
- `city` — high-cardinality categorical (100+ distinct cities); dropped for simplicity in favor of `city_development_index`, which is a numeric proxy for the same signal.
- `gender` — dropped (also has missing values in the raw data).

4. Initial exploration

`df.shape`, `df.info()`, `df.describe()`, and `df.describe(include='object')` are used to inspect data types, ranges, and missing-value counts. Key findings from `df.isna().sum()` before cleaning:

Column	Missing
enrolled_university	417
education_level	512
major_discipline	3,125
experience	70
company_size	6,560
company_type	6,774
last_new_job	463
target	2,129 (expected — these are the test rows)

5. Handling missing values

```
df['company_type'] = df['company_type'].fillna('No Company')
df['enrolled_university'] = df['enrolled_university'].fillna('no_enrollment')
df['education_level'] = df['education_level'].fillna(df['education_level'].mode()[0])
df['major_discipline'] = df['major_discipline'].fillna('Unknown')
df['experience'] = df['experience'].fillna(df['experience'].mode()[0])
df['company_size'] = df['company_size'].fillna('Unknown')
df['last_new_job'] = df['last_new_job'].fillna(df['last_new_job'].mode()[0])
```

Strategy used per column:

- **Semantic fill** (`company_type` → 'No Company', `enrolled_university` → 'no_enrollment', `major_discipline` / `company_size` → 'Unknown'): a missing value here plausibly *means* "not applicable," so a dedicated category is more honest than a statistical fill.
- **Mode fill** (`education_level`, `experience`, `last_new_job`): missingness looks closer to "not reported," so the most frequent value is used as a neutral default.

After this step, `df.isna().sum()` shows zero missing values in every feature column (only `target` still has nulls, which is expected for the 2,129 test rows).

Note on methodology: these fills (especially the mode calculations) are computed on the **combined** train+test frame. For a stricter, leakage-free pipeline, `mode()` should be computed on the training split only and then applied to test/validation data.

6. Encoding categorical features

Each categorical feature is inspected with `.unique()` before deciding on an encoding strategy:

Feature	Encoding	Mapping
education_level	Ordinal map	Primary School=1, High School=2, Graduate=3, Masters=4, Phd=5
experience	Numeric cleanup	'<1' → 0, '>20' → 21, rest cast to float
last_new_job	Numeric cleanup	'>4' → 3, 'never' → 0, rest cast to float
company_size	Ordinal map	Unknown=0, <10=1, 10-49=2, 50-99=3, 100-500=4, 500-999=5, 1000-4999=6, 5000-9999=7, 10000+=8
relevent_experience	Binary map	Has relevent experience=1, No relevent experience=0
company_type , enrolled_university , major_discipline	LabelEncoder	Arbitrary integer codes assigned alphabetically

```
CustomizedELevel = {'Primary School': 1, 'High School': 2, 'Graduate': 3, 'Masters': 4, 'Phd': 5}
df['education_level'] = df['education_level'].map(CustomizedELevel)

df['experience'] = df['experience'].replace({'<1': 0, '>20': 21}).astype(float)
df['last_new_job'] = df['last_new_job'].replace({'>4': 3, 'never': 0}).astype(float)

size_map = {'<10': 1, '10/49': 2, '50-99': 3, '100-500': 4, '500-999': 5,
          '1000-4999': 6, '5000-9999': 7, '10000+': 8, 'Unknown': 0}
df['company_size'] = df['company_size'].map(size_map)

df['relevent_experience'] = df['relevent_experience'].map(
    {'Has relevent experience': 1, 'No relevent experience': 0}
)

LE = LabelEncoder()
df['company_type'] = LE.fit_transform(df['company_type'])
df['enrolled_university'] = LE.fit_transform(df['enrolled_university'])
df['major_discipline'] = LE.fit_transform(df['major_discipline'])
```

Note on methodology: `education_level`, `experience`, `last_new_job`, and `company_size` are genuinely ordinal, so integer mapping is appropriate. `company_type`, `enrolled_university`, and `major_discipline` are **nominal** (no inherent order), so `LabelEncoder` introduces an artificial ordering that a linear model (Logistic Regression) could misinterpret as meaningful magnitude. One-hot encoding (`OneHotEncoder`, already imported but unused) would be the more correct choice for these three columns.

7. Splitting features/target and train/test

```
df_train_clean = df[df['source'] == 'train'].drop('source', axis=1)
df_test_clean = df[df['source'] == 'test'].drop(['source', 'target'], axis=1)

x = df_train_clean.drop('target', axis=1)
y = df_train_clean['target']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)
```

- The combined frame is split back into the labeled training set (`df_train_clean`) and the unlabeled inference set (`df_test_clean`).
- `x / y` are built from the labeled data only.
- An 80/20 **stratified** split (`stratify=y`) preserves the ~25%/75% class balance in both the training and

holdout sets.

- Resulting shapes: `x_train` (15,326, 10), `x_test` (3,832, 10).

8. Feature scaling

```
standard = StandardScaler()
x_train_scaled = standard.fit_transform(x_train)
x_test_scaled = standard.transform(x_test)
```

`StandardScaler` is **fit only on `x_train`** and then applied to `x_test` — this is the correct order of operations and avoids leaking test-set statistics into training. Scaling matters most for Logistic Regression (distance/weight-based) and helps Naive Bayes' Gaussian assumption; it's less critical for the Decision Tree, which is scale-invariant, but doesn't hurt.

9. Evaluation helper

```
results = {}
roc_data = {}

def evaluate_model(name, model, x_test, y_test, color='Greens'):
    y_pred = model.predict(x_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results[name] = {'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1 Score': f1}
    print(classification_report(y_test, y_pred))
    # + confusion matrix plot
```

A single reusable function computes accuracy/precision/recall/F1, prints a full classification report, plots a confusion matrix, and stores results in a shared `results` dict so all three models can be compared later in one table.

10. Models trained

All three models use `class_weight='balanced'` to compensate for the ~25%/75% class imbalance (recall/precision on the minority "looking for job change" class matters more than raw accuracy for this use case).

Logistic Regression

```
logreg = LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
logreg.fit(x_train_scaled, y_train)
```

Decision Tree

```
dt = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42, class_weight='balanced')
dt.fit(x_train_scaled, y_train)
```

`max_depth=7` caps tree depth to control overfitting. The fitted tree is visualized with `plot_tree(...)`.

Gaussian Naive Bayes

```
gnb = GaussianNB()
gnb.fit(x_train_scaled, y_train)
```

For each model, the notebook prints **training-set** metrics (to gauge over/underfitting) and then calls `evaluate_model(...)` on the **held-out test set**, plus an ROC curve and AUC score via `roc_curve` / `roc_auc_score`.

11. Model comparison

```
results_df = pd.DataFrame(results).T.sort_values('F1 Score', ascending=False)
results_df[['Accuracy', 'Precision', 'Recall', 'F1 Score']].plot(kind='bar', ...)
```

Results are collected into a single sorted table and visualized as a grouped bar chart (see `REPORT.md` for the numbers).

12. Feature importance

```
importances = pd.Series(dt.feature_importances_, index=x_train.columns).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index, palette='Oranges')
```

Extracts and plots Gini-based feature importances from the fitted Decision Tree — see `REPORT.md` for the ranked list.

13. Insights on raw data

```
df_raw = pd.read_csv('aug_train.csv')
for col in ['education_level', 'company_type', 'relevant_experience', 'enrolled_university']:
    print(df_raw.groupby(col) ['target'].mean().sort_values(ascending=False))
```

The notebook re-reads the **original, unencoded** `aug_train.csv` to compute human-readable "target rate" (proportion of candidates looking for a job change) grouped by category, then visualizes two of these groupings as bar charts and prints overall dataset statistics (total candidates, number job-seeking, overall rate).

Reproducibility notes

- `random_state=42` is used consistently across `train_test_split`, `LogisticRegression`, and `DecisionTreeClassifier`, so results should be reproducible run to run given identical input data and library versions.
- `GaussianNB` has no random component and needs no seed.